

U3 · Aula 1 — Playwright Avançado

Locators, Auto-waiting e Network Mocking

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Fundamentos:** por que Playwright venceu, locators, auto-waiting
- **Parte 2 — Rede e estado:** network mocking, HAR, storageState, fixtures
- **Parte 3 — Prática guiada:** login e busca no CineFav Web

Depois desta aula: screencast — eu resolvendo os specs passo a passo.

Objetivos de aprendizagem

- **Escolher** locators estáveis (role, text, test-id) e justificar a hierarquia
- **Explicar** por que auto-waiting elimina `sleep` (e flakiness junto)
- **Mockar** a rede com `route()` — backend sob controle do teste
- **Reutilizar** autenticação com `storageState` (login 1x por suíte)
- **Completar** os specs de login e busca num app real (CineFav Web)

Parte 1 — Fundamentos

Por que Playwright · locators · auto-waiting · contexts

Por que Playwright tomou o mercado?

Em 2026, Playwright substituiu Selenium e Cypress em times sérios.

Razão	Detalhe
CDP (não WebDriver)	Controle granular sobre browser
Browser contexts isolados	Paralelismo sem conflito de estado
Auto-waiting nativo	Zero <code>sleep</code> nos testes
Locators recomendados	<code>role</code> , <code>text</code> , <code>test-id</code>
Cross-browser	Chromium, Firefox, WebKit
Trace viewer	Debug sem reproduzir local

Locators recomendados — a hierarquia correta

```
//  Role + nome (preferido)  
page.getByRole('button', { name: 'Entrar' });  
page.getByRole('link', { name: 'Sobre nós' });  
  
//  Texto visível  
page.getByText('Welcome back');  
  
//  Test-id (estável entre refactors)  
page.getByTestId('submit-button');  
  
//  Label / placeholder  
page.getByLabel('Senha');  
page.getByPlaceholder('Email');  
  
//  Frágil — quebra com qualquer refactor de CSS/DOM  
page.locator('div.container > .card:nth-child(2) span.title');
```

Auto-waiting — como funciona

Playwright aguarda automaticamente antes de cada ação:

1. Elemento **anexado** ao DOM
2. Elemento **visível** na tela
3. Elemento **estável** (sem animação em andamento)
4. Elemento **habilitado** (não disabled)
5. Elemento **clicável** (não coberto por outro elemento)

```
// Não precisa de waitFor manual  
await page.getByRole('button', { name: 'Entrar' }).click();  
  
// Web-first assertions também aguardam  
await expect(page.getByText('Welcome')).toBeVisible({ timeout: 5000 });
```

Se ainda usa `waitForTimeout`, há algo errado.

Browser context — isolamento de estado

```
const browser = await chromium.launch();

// Cada context tem cookies, storage e history próprios
const ctx1 = await browser.newContext({ locale: 'pt-BR' });
const ctx2 = await browser.newContext({ locale: 'en-US' });

const page1 = await ctx1.newPage();
const page2 = await ctx2.newPage();

// Rodam em paralelo sem interferência
await Promise.all([
  page1.goto('/'),
  page2.goto('/'),
]);
```

1 browser → N contexts → M pages. Paralelismo sem conflito.

Parte 2 — Rede e estado

`route()` · HAR replay · `storageState` · fixtures

Network mocking com `route()`

```
test('exibe mensagem de erro quando API falha', async ({ page }) => {
  await page.route('**/api/checkout', async (route) => {
    await route.fulfill({
      status: 500,
      contentType: 'application/json',
      body: JSON.stringify({ error: 'Internal Server Error' }),
    });
  });

  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Pagar' }).click();

  await expect(page.getByText('Erro. Tente novamente.')).toBeVisible();
});
```

Testa o comportamento da UI em cenários de falha sem precisar de backend quebrando.

HAR replay — tráfego determinístico

Captura uma sessão real e repete deterministicamente nos testes.

```
// Passo 1: capturar
const context = await browser.newContext({
  recordHar: { path: 'session.har' },
});

// Passo 2: replay nos testes
await page.routeFromHAR('session.har', {
  url: '**/api/**',
  notFound: 'fallback', // requests não no HAR passam normalmente
});
```

Útil para testar fluxos complexos contra dados reais sem backend disponível.

storageState — reutilizar autenticação

Problema: login em cada teste = lento (3s × 200 testes = 10min só de login).

```
// global-setup.ts — executa 1 vez antes de toda suíte
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@email.com');
  await page.getByLabel('Senha').fill('secret');
  await page.getByRole('button', { name: 'Entrar' }).click();
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: 'auth.json' });
  await browser.close();
}
```

```
// playwright.config.ts
export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  use: { storageState: 'auth.json' },
});
```

Fixtures customizadas — composição limpa

```
// fixtures.ts
export const test = base.extend<{ loggedInPage: Page }>({
  loggedInPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('user@email.com');
    await page.getByLabel('Senha').fill('secret');
    await page.getByRole('button', { name: 'Entrar' }).click();
    await page.waitForURL('/dashboard');
    await use(page);
  },
});

// test-checkout.spec.ts
test('completa compra', async ({ loggedInPage }) => {
  await loggedInPage.goto('/produtos');
  // ...
});
```

Pitfalls comuns

- ✗ `waitForTimeout` — workaround, não solução. Acha o root cause.
- ✗ **Locators por hierarquia CSS** — frágeis, quebram com qualquer refactor visual.
- ✗ **Mockar tudo** — testes desacoplam da realidade do backend.
- ✗ **Sem `storageState`** — suíte fica lenta com logins repetidos.
- ✗ `retries: 3` sem investigar flake — mascara problema real.

Resumo

- **Locators por role/texto/test-id** → estáveis e semânticos
- **Auto-waiting** → zero `sleep`, Playwright aguarda a condição
- `route()` → mockar APIs sem backend; **HAR** → replay determinístico
- `storageState` → login 1 vez por suíte, não por teste
- **Fixtures** → composição limpa de contextos reutilizáveis

Parte 3 — Prática guiada

Os specs que você vai completar (com screencast)

A prática — app CineFav Web

- **SPA de filmes real** — login, busca, favoritos, detalhe (React + Vite)
- **Mesmo produto do CineFav mobile** — mesmos `data-testid`, mesmo catálogo
- **Catálogo via HTTP** — `GET /api/movies.json` → dá pra interceptar com `route()`
- **Roda offline, sem token** — dados mockados, determinístico
- **Você escreve só os testes** — o app já vem pronto

Setup em 3 passos

```
# 1. entre na pasta da prática (repo da disciplina)
cd exercicios/02-lab-web-pwa-playwright/pratica
ls tests/e2e      # confirma: auth.setup.ts, 01-login.spec.ts, ...

# 2. instale dependências + browser
npm install
npx playwright install chromium

# 3. rode a suíte (builda o app e testa contra o preview)
npm run test:e2e
```

App no browser: `npm run dev` → localhost:5173 · login aluno@puc.br / 1234

Prática 1 — Login (01-login.spec.ts)

```
test('2. login com credenciais válidas leva à lista de filmes', async ({ page }) => {  
  await page.goto('/login');  
  
  await page.getByTestId('login-email-input').fill('aluno@puc.br');  
  await page.getByTestId('login-password-input').fill('1234');  
  await page.getByTestId('login-submit-button').click();  
  
  await expect(page.getByTestId('movielist-screen')).toBeVisible();  
});
```

- **Modelo resolvido** — no screencast eu rodo e explico linha a linha
- `auth.setup.ts` (também resolvido) salva o **storageState**: login 1x, specs 02+ já entram logados

Prática 2 — Busca com mock (02-busca-mock.spec.ts)

```
// TODO: intercepte '**/api/movies.json' com route.fulfill  
// TODO: injete um filme inventado (id 999)  
// TODO: busque e verifique search-result-999 visível
```

- **Você completa os TODOs** — fulfill, abort, unroute, retry
- **⚠ Pitfall que você vai encontrar:** o Service Worker da PWA responde do cache — requisição que o SW responde **não passa pelo route()** .
O spec já traz `test.use({ serviceWorkers: 'block' })` — entenda o porquê.

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Repo da prática** — link no Canvas, specs-guia prontos
- **Seu ritmo** — pause, rode, compare com o resultado
- **Travou?** — `npm run test:e2e:ui` abre o modo UI (o melhor debugger do Playwright)

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith